



UEFS

**UNIVERSIDADE ESTADUAL
DE FEIRA DE SANTANA**

DEPARTAMENTO DE TECNOLOGIA
TEC499 - SISTEMAS DIGITAIS
2025.2

API para controlador gráfico

DOCENTE: ANFRANSERAI MORAIS DIAS
DISCENTES: DANIEL PORTO BRAZ
DISNEY CARVALHO ANDRADE FILHO
LEVI NOGUEIRA VASCONCELOS

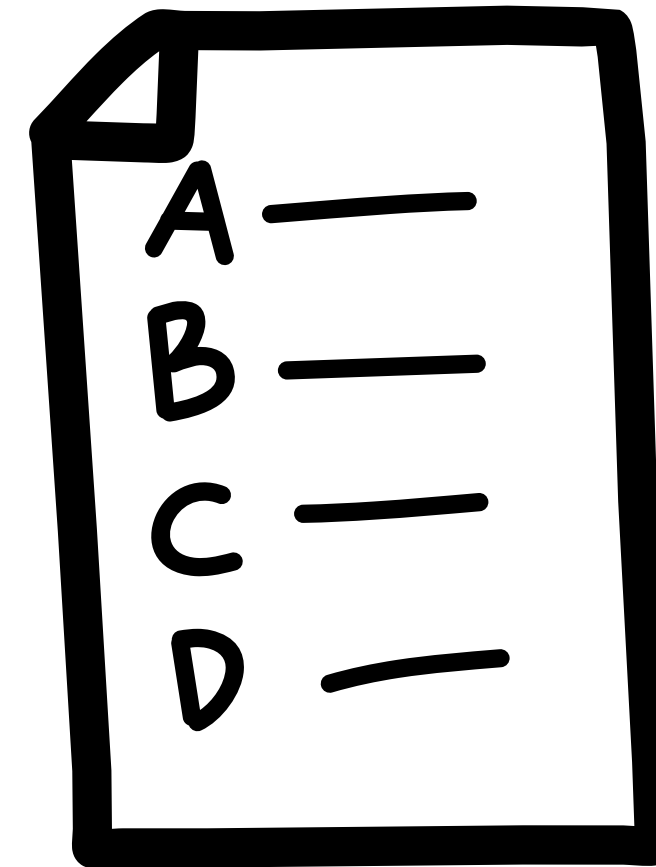
Roteiro

- Descrição e Requisitos do Projeto
- ISA
- Conexão HPS - FPGA
- API
- Exemplos de Execução



Descrição e Requisitos

- API escrita em assembly;
- Comandos da ISA do coprocessador devem ser implementados na API;
- A imagem deve ser lida a partir de um arquivo e transferida para o coprocessador;
- O coprocessador deve ser compatível com o processador ARM (*Hard Processor System - HPS*);
- As imagens são representadas em escala de cinza e cada pixel deverá ser representado por um número inteiro de 8 bits.





ISA

- **Controlador gráfico para zoom in e zoom out de imagens;**
- **Instruções de 29 bits;**
- **Quatro flags: Done, Error, Max zoom e Min zoom.**

Figura 1 — Estrutura da instrução

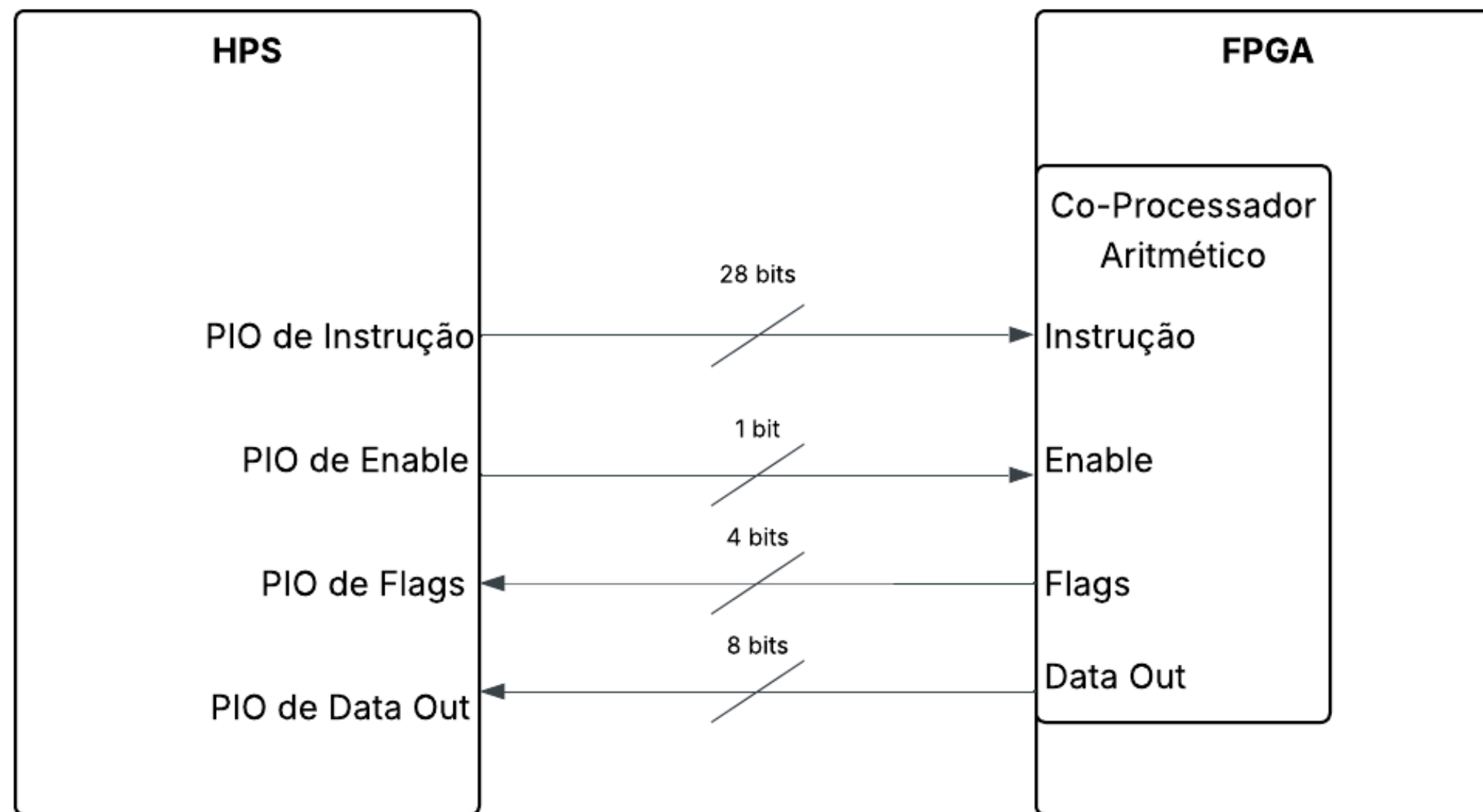


Tabela 1 — Tabela dos Op codes

	Refresh	Load	Store	NH (In)	PR	NH (Out)	MB	Reset
Op code	000	001	010	011	100	101	110	111

Conexão HPS – FPGA

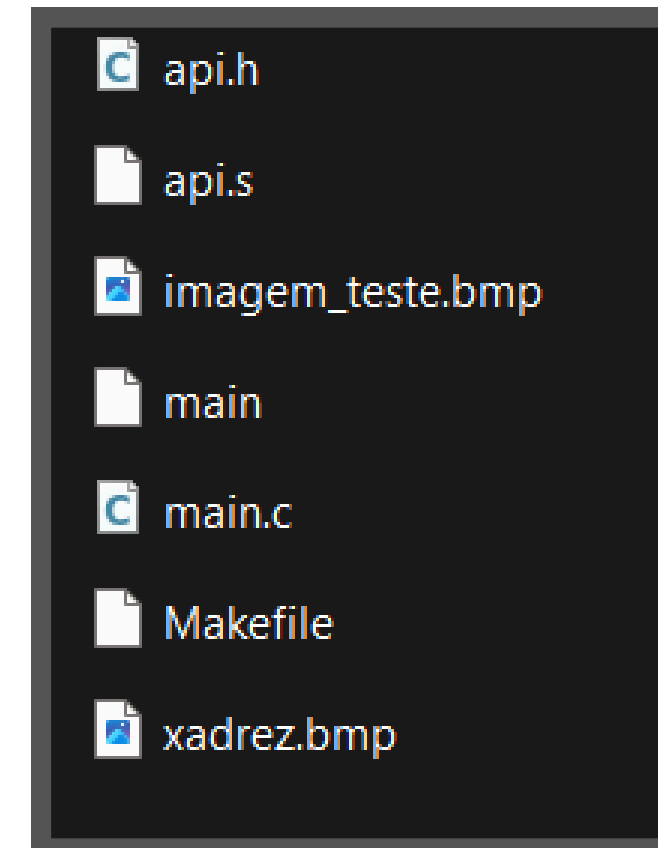
Figura 2 — Diagrama da conexão HPS-FPGA



- **Co-Processador desenvolvido por Maíke de Oliveira;**
- **Barramento: Lightweight;**
- **PIOs de saída:**
 - **Instrução (29 bits);**
 - **Enable (1 bit).**
- **PIOs de entrada:**
 - **Flags (4 bits);**
 - **Data Out (8 bits).**

API – Estrutura Geral

- **Arquivos:**
 - **Makefile:** script para compilação do projeto;
 - **main.c:** Interface do usuário e código cliente (alto nível);
 - **api.h:** arquivo cabeçalho que define interface da API;
 - **api.s:** funções em Assembly ARMv7-A (baixo nível).



API – Arquitetura ARMv7

- A arquitetura ARMv7 utiliza um conjunto de 16 registradores de uso geral, além de registradores especiais responsáveis por controle de fluxo e pilha.
- No ARMv7, uma syscall é realizada carregando o número da chamada no registrador r7 e seus argumentos nos registradores r0–r6, seguida da instrução svc #0

API – Arquitetura ARMv7

Tabela de Registradores

Registrador	Nome	Função
r0 - r3	Argumento / Retorno	Recebe os parâmetros de r0 a r3 e realiza o retorno da função em r0
r4 - r11	Calle-saved	Valores preservados entre as funções
r7		Registrador que carrega o número da syscall
r12	IP / Scratch	Registrador Temporário
r13	SP (Stack Pointer)	Armazena o endereço do topo da pilha, esta que é utilizada para guardar memória entre chamadas de funções
r14	LR (Link Register)	Armazena o endereço de retorno da função
r15	PC (Program Counter)	Indica o endereço para a próxima instrução (8 bytes p/ endereço no ARMv7)

API – Arquitetura ARMv7

Chamadas de Sistema		
Chamada	Número	Protótipo em C
EXIT	1	void _exit(int status);
READ	3	ssize_t read(int fd, void *buf, size_t count);
OPEN	5	int open(const char *pathname, int flags,
CLOSE	6	int close(int fd);
LSEEK	19	off_t lseek(int fd, off_t offset, int whence);
MMAP2	192	void *mmap2(void *addr, size_t length, int prot, int flags, int fd, off_t pgoffset);
MUNMAP	19	int munmap(void *addr, size_t length);

API – Funções e Erros

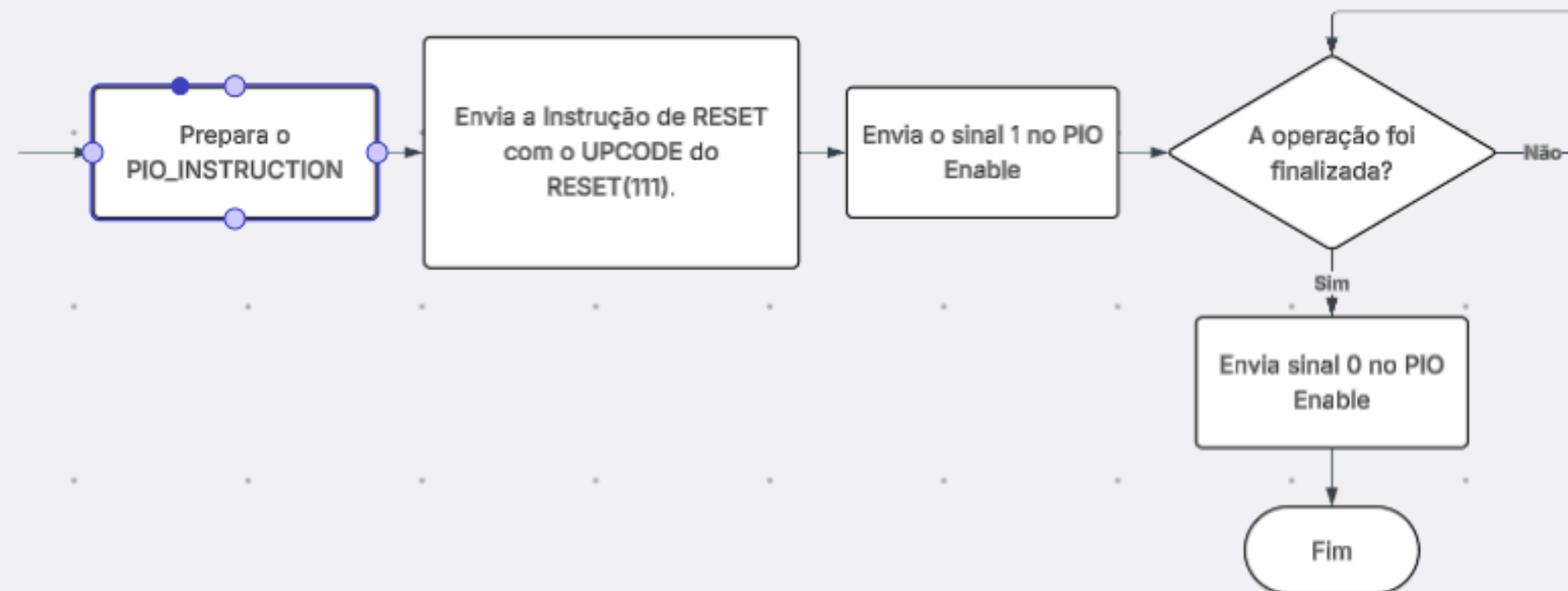
API – Parâmetros

	NH(In), PR, NH(Out), MB	Load	Store	API_INIT API_FINALIZE
Protótipo	int api_mblocos(void);	int api_load(int mem, int address);	int api_store(const char *file_path);	int api_init(void);
Parâmetros	N/A	int mem; int adress;	const char *file_path;	N/A
Retorno	void()	int pixel;	void()	void()



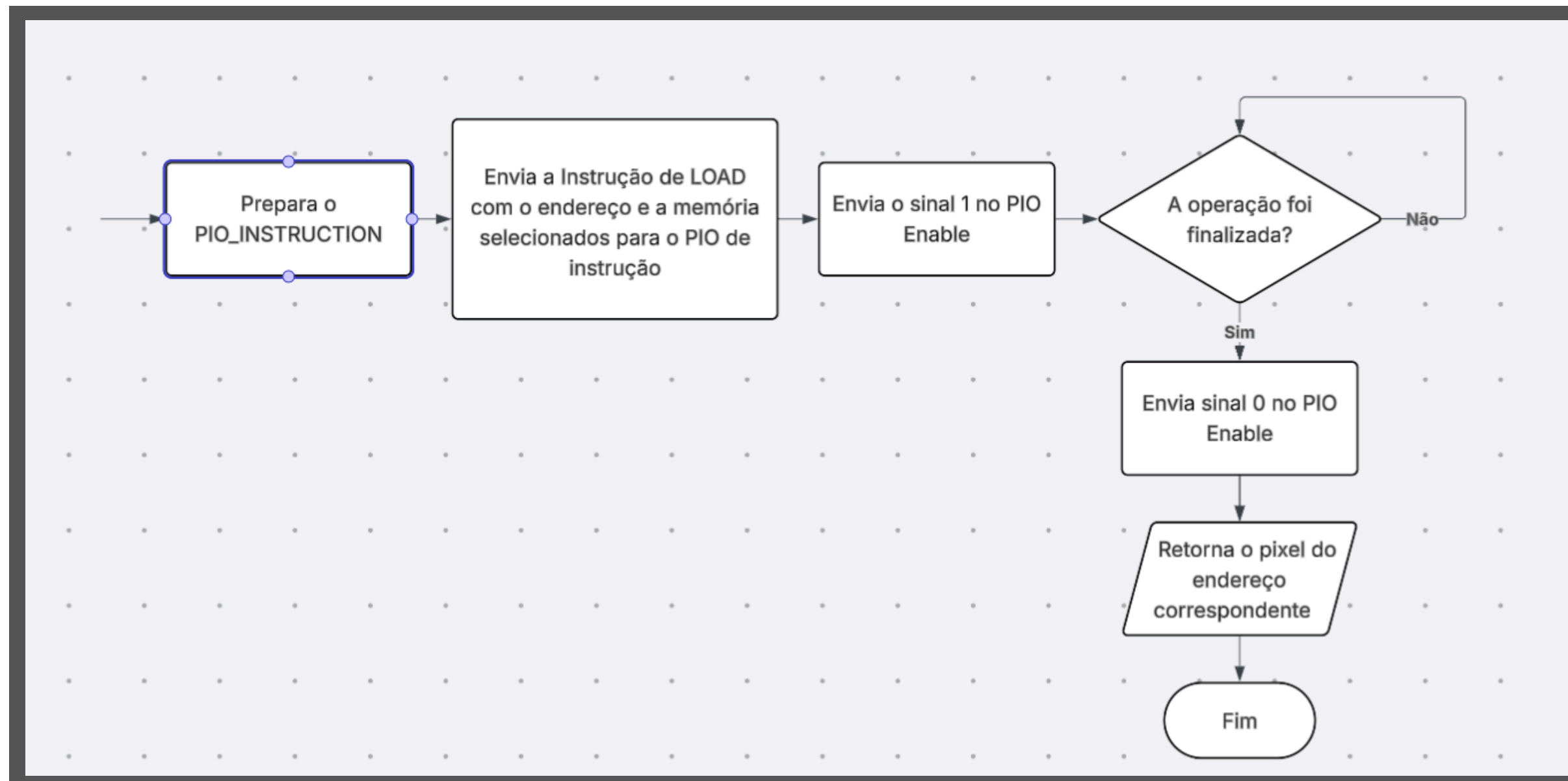
API – Funções e Erros

API – RESET



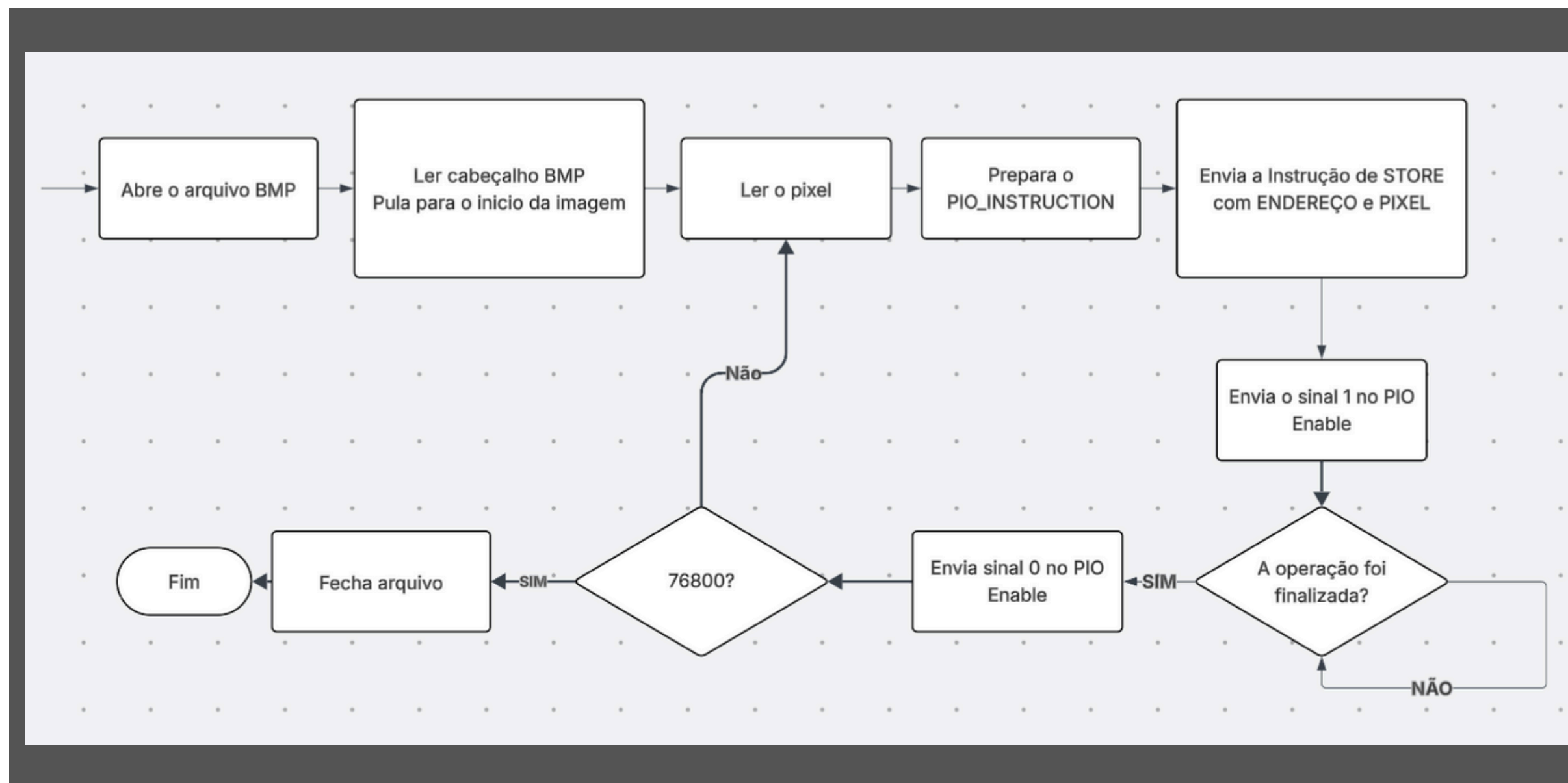
API – Funções e Erros

API – LOAD



API – Funções e Erros

API – STORE



Exemplos de Execução

- **Imagem Exibida inicialmente**



- **Zoom-Out**



- **Zoom-In**



- **Imagem carregada com Store**

